



Nombre:

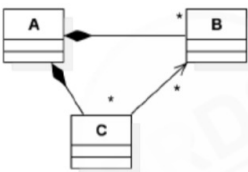
Grupo:

Pregunta 1 (2 puntos):

Contestar V/F las siguientes cuestiones, teniendo en cuenta que una respuesta incorrecta invalida una correcta.

- [] Todo mapa puede convertirse en un set si a la clase de objetos que contiene se le dotan de los operadores de comparación necesarios.
- [] Los datos dentro de un nodo de un árbol B deben estar necesariamente ordenados.
- [] Dada la siguiente definición de un multimapa: `multimap<Fecha, Reserva, greater<Fecha> > reservas;` la clase `Fecha` necesita la definición del operador `<`.
- [] La operación de inserción en una lista simplemente enlazada en la posición anterior al nodo apuntado por el iterador `it` es una operación en tiempo constante: `bool ListaEnlazada<T>::insertarAnt(const Iterador<T> &it, const T &dato);`
- [] Es más eficiente un quadtree que una malla regular a nivel de consumo de memoria para almacenar todos los códigos postales de EEUU.
- [] El número de accesos a disco que se necesita para localizar un dato mediante un sistema de ficheros con un índice simple, depende o es directamente proporcional al tamaño de dicho índice.
- [] Mediante la siguiente definición: `int **p;` `p` puede ser

- una matriz bidimensional, un vector o nada de lo anterior.
- [] Una cola con prioridad siempre tendrá un `push()` en $O(1)$ independientemente de la implementación concreta (vectores, vector de listas o listas de listas)
- [] Averiguar las aristas de entrada a un nodo en un grafo dirigido implementado mediante listas invertidas es más costoso que implementado sobre una matriz de adyacencia.
- [] Es peligroso usar un vector dinámico de objetos (no de punteros) si otra clase va a tener una asociación con estos objetos. Mirar el siguiente ejemplo, en el que la relación de composición entre A y B se implementa como: `vector<B>`.



Pregunta 2 (2 puntos):

Los conjuntos disjuntos pueden usarse para segmentar imágenes, de forma que cada conjunto disjunto representará una zona formada por un conjunto de píxeles contiguos de similares características (p.ej. del mismo color). En cada paso el algoritmo toma dos píxeles vecinos p_1 y p_2 y realiza la siguiente operación:

```
// Si dos píxeles pertenecen a dos conjuntos distintos pero son similares entre sí
if (buscar( $p_1$ ) != buscar( $p_2$ ) && similar( $p_1$ ,  $p_2$ )) {
    unir(buscar( $p_1$ ), buscar( $p_2$ ));
}
```

La matriz I mostrada abajo representa una mini imagen de 4x4 píxeles con niveles de gris (0-> negro, 1-> blanco). Utilizar conjuntos disjuntos para separar la imagen en zonas de intensidad similar. Entenderemos que dos píxeles son similares si la diferencia entre sus intensidades es menor o igual a 0.2 (en valor absoluto). Los píxeles están numerados de 0 a 15 empezando en la esquina superior izquierda.

La matriz D guarda los conjuntos disjuntos, donde inicialmente cada pixel pertenece a un conjunto distinto cuyo representante es él mismo, como se ilustra al final del enunciado. Mostrar el estado de la matriz D tras cada pasada completa a todos los píxeles realizando la comprobación mostrada arriba. En cada pasada se procesará la matriz siguiendo la numeración de los píxeles. En cada paso comparar un pixel con su vecino de abajo primero, y con su vecino a la derecha después. Terminar cuando tras una pasada completa no se produzca ninguna unión de conjuntos. Mostrar finalmente los conjuntos obtenidos en forma de árboles.

Nota: para simplificar el ejercicio, no usar compresión de caminos en las búsquedas ni optimizaciones en la unión (unión del conjunto con un árbol de menor altura al de mayor altura).

I				D			
0.3	0.6	0.5	0.2	0	1	2	3
0.7	0.6	0.4	0.1	4	5	6	7
0.6	0.5	0.1	0.1	8	9	10	11
0.5	0.5	0.2	0.2	12	13	14	15



Esta obra pertenece a ERDS bajo la licencia de Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional

Pregunta 3 (3 puntos):

- (a) Es posible que casualmente un ABB cumpla las propiedades de equilibrio exigidas a un AVL. Implementar la función `compatibleAVL()` para que devuelva un valor `TRUE` si se detecta que dicho árbol está equilibrado como un AVL. Esta función no lo convierte en AVL, pero sí ayudará a su construcción añadiendo el valor `altura` del nodo del árbol. Implementar dicha función para que devuelva el valor `TRUE/FALSE` mientras que rellena para todo el árbol la `altura` que está inicializada a 0.

```
template <typename T>
class NodoABB {
    Nodo<T> *izq, *der;
    T dato;
    int altura;
};

...

template<typename T>
bool ABB<T>::compatibleAVL();
```

- (b) Implementar la función de búsqueda de un dato en una tabla hash con cubetas de tamaño constante `tamaCub` en una tabla con `nCub` cubetas, teniendo en cuenta los posibles desbordamientos. La función devuelve la posición de la cubeta donde se encuentra el dato y en caso de no estar devuelve -1.

```
struct Cubeta{
    unsigned tamalog; //tamaño lógico
    unsigned buffer[tamaCub]; //buffer de datos
    bool desbordada; //indicador de desbordamiento
};

class THashCubetas{
    vector<Cubeta> tabla;
};

int nCub;
unsigned hash(int dato, int intento);
public:

    int buscar(unsigned dato);
    ...
};
```

Pregunta 4 (3 puntos):

La Universidad de Jaén ha decidido apuntarse también a la moda de las criptomonedas inventando la suya propia, el ujacoin, que podrá utilizarse para pagos dentro del campus. Para garantizar la seguridad en las transacciones (básicamente transferencias de una cuenta a otra) existe un servidor que guarda los bloques de transacciones y las firma mediante un sistema de minado similar al utilizado por bitcoin. Las clases involucradas son las siguientes:

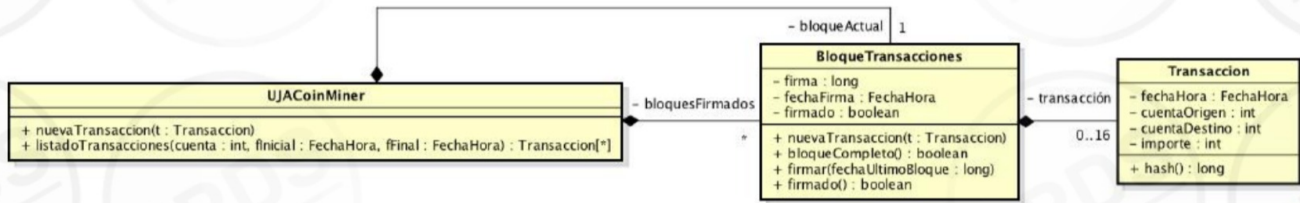
Transaccion representa una transferencia de dinero entre dos cuentas ujacoin. Dispone de un constructor donde se indican los datos de la transferencia (la fecha y hora es tomada automáticamente del reloj del sistema). La operación `hash()` devuelve un hash calculado como $H_T = djb2(fechaHora.convertirALong(), cuentaOrigen, cuentaDestino, importe)$. La función `djb2()` es descrita más abajo.

BloqueTransacciones representa un bloque de 16 de transacciones. La operación `nuevaTransaccion()` permite añadir nuevas transacciones (lanza una excepción si el bloque está completo, es decir si tiene ya 16 transacciones). La operación `firmar()` es la que firma el bloque una vez está completo (lanza una excepción en caso de que no lo esté). El algoritmo de firmado se describe más abajo. Además se anota la fecha y hora de firmado.

UJACoinMiner representa el servidor de minado. Tiene una operación para añadir transacciones al bloque actual. Cuando el bloque actual está completo se firma y se pasa a la lista de bloques firmados, creando un nuevo bloque vacío. Además incluye una operación para listar transacciones (solo las firmadas) a partir de dos fechas dadas donde la cuenta indicada sea origen o destino de la transferencia.

Para generar el hash de un conjunto de números usaremos una versión de la función `djb2()` que estudiamos en el tema de dispersión, pero tomando un vector de long como entrada en lugar de un vector de caracteres: `long djb2(vector<long> datos)` (no implementar). Tampoco implementar la clase `FechaHora`. Supondremos que tiene un constructor por defecto que toma la fecha y hora del sistema, la operación `convertirALong()` y los operadores de comparación correspondientes.

La firma de un bloque se realizará de la siguiente manera. En primer lugar calculando el hash del bloque como: $H_{Bloque} = djb2(F_{UltimoBloque}, H_{T0}, H_{T1}...H_{T15})$, donde $F_{UltimoBloque}$ es la firma del último bloque de la lista de bloques firmados. Entonces la firma del bloque será un long $F_{Bloque} = x$ que verifique: $0 < djb2(H_{Bloque}, x) < 11117$. Este valor hay que buscarlo a base de ensayo y error (pero se garantiza que existe).



powered by Astal



Esta obra pertenece a ERDS bajo la licencia de Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional